

```

/**
 * @class invalid_peek_error
 * @brief Exception thrown when attempting to peek into an empty stack.
 */

/**
 * @class invalid_pop_error
 * @brief Exception thrown when attempting to pop from an empty stack.
 */

/**
 * @class Stack
 * @brief A class template implementing the Stack ADT using a linked list.
 *
 * The Stack provides LIFO (last-in, first-out) access to elements.
 * It supports standard stack operations: push, pop, peek, and size.
 *
 * @tparam StackElementType The type of elements stored in the stack.
 */
{
    /**
     * @struct Node
     * @brief Represents a single node in the linked list.
     */

    /**

```

* @brief Constructs a new Node.
* @param value The data value for this node.
* @param nextNode Pointer to the next node (default nullptr).
*/

/**

* @brief Default constructor. Initializes an empty stack.
* @post The stack is empty and count = 0.
*/

/**

* @brief Destructor. Frees all dynamically allocated nodes.
* @post All nodes are deleted; stack is empty.
*/

/**

* @brief Adds a new item to the top of the stack.
* @param item The value to add to the stack.
* @post item is stored at the top of the stack.
*/

/**

* @brief Removes and returns the top item from the stack.
* @return The value at the top of the stack.
* @throws invalid_pop_error if the stack is empty.
* @post The top item is removed; stack size decreases by one.

*/

/**

* @brief Returns the value at the top of the stack without removing it.

* @return A copy of the top item.

* @throws invalid_peek_error if the stack is empty.

*/

/**

* @brief Returns the number of elements currently in the stack.

* @return The number of elements in the stack.

*/

/**

* @brief Determines whether the stack is empty.

* @return True if the stack has no elements; false otherwise.

*/